

Introduction to Programming with Scientific Applications (Spring 2025)

Final project

Study ID	Name	% contributed
202307989	Mikkel Moth Billing	100%

max 3 students

Briefly state the contributions of each of the group members to the project

I did all the coding and the report writing.

Declaration on Generative Artificial Intelligence (see <https://studerende.au.dk/en/gai>)

I used generative artificial intelligence (GAI) to complete this project. I used ChatGPT and Copilot to help with bugfixing and to get a better understanding of the processes of machine learning.
I have stated what i have used GAI for in the report. I have not copied code just used it for inspiration.

Note on plagiarism

Since the evaluation of the project report and code will be part of the final grade in the course, **plagiarism in your project handin will be considered cheating at the exam.** Whenever adopting code or text from elsewhere you must state this and give a reference/link to your source. It is perfectly fine to search information and adopt partial solutions from the internet – actually, this is encouraged – but always state your source in your handin. Also discussing your problems with your project with other students is perfectly fine, but remember each group should handin their own solution. If you are in doubt if your solution will be very similar to another group because you discussed the details, please put a remark that you have discussed your solution with other groups. For more Aarhus University information on plagiarism, please visit <http://library.au.dk/en/students/plagiarism/>.

1 Introduction

For this project I had to look at Image Classification, by creating a simple neural network. This neural network is used to classify images of handwritten digits from the MNIST database. These images are given in 28x28 grayscale images, where each pixel is a number between 0 and 255. I then use linear classification using a 784x10 matrix and a bias vector of size 10, to guess the number. From this guess we can calculate a cost, which is the mean square error of the guess and the actual number. By minimizing this cost function using linear algebra, we can update the weight matrix and bias vector. The weights can then be visualized and the cost and accuracy can be plotted, to show how the neural network evolves.

2 Code

Throughout the code I have used the decorator `@time_it`, that we have from the lectures, to time the functions used.

2.1 Reading Data

For getting the raw data out of the MNIST files I had to first understand the format of the .gz files. For this I have used ChatGPT to help me understand how to read the bytes in the file, to get the magic numbers. Then as we can see at <https://github.com/sunsided/mnist> the first 4 bytes of the files are the magic number, which is 2049 for label files and 2051 for images files. Then the files contains 4 bytes depicting the size of the file, additionally the image files contains 8 bytes for the size the images. After that we can read the raw data.

To read the bytes I open the files with `gzip.open` then read the files with `.read`. Then we can plot the images with their correct label, to see that we have read the data correctly, since the first 5 numbers of `t10k-labels.idx1-ubyte.gz` are [7, 2, 1, 0, 4] as seen in figure 1.

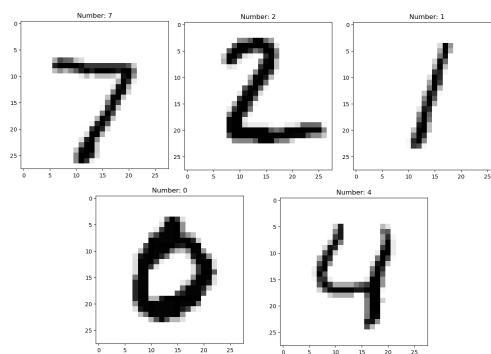
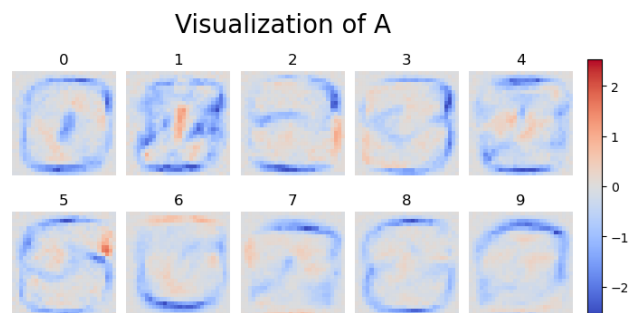


Figure 1: The first 5 images of the MNIST test set.

I also coded a way to read and write JSON files so that the weight matrix and bias vector can be saved. This is done with the json library. By doing this we can visualize the weights and biases from MNIST as seen in figure 2.



Figur 2: The weights of the MNIST neural network.

2.2 Linear Algebra

Since numpy is not allowed in the project, I had to write linear algebra functions to use on list vectors and matrices. These include vector addition, subtraction, and scalar multiplication. For these I wrote assertion tests to check if the given vectors are the same length, and that the scalar is actually a number.